WTL Mini-project

Project Report: StockPilot Inventory Management System

Project Title	StockPilot Inventory Management System
Project Type	Dynamic web application for inventory operations
Repository	https://github.com/vedant27-lab/stockpilot.git
Prepared For	WTL Mini-project submission

Problem Statement of Mini-Project

Retail stores and small businesses often struggle to maintain accurate stock records, track incoming and outgoing inventory, identify reorder requirements, and coordinate access among team members. Manual methods lead to stock mismatch, delayed updates, and poor visibility. This project solves that problem by providing a dynamic inventory management web application with analytics, stock movement tracking, and controlled access sharing.

Objective

To learn web technologies like HTML, CSS, JS, JSP/SERVELET/ASP.NET/PHP, MYSQL, XML, etc. to implement dynamic web application.

The project applies these concepts by building a responsive inventory workspace using HTML, CSS, JavaScript, Node.js, and JSON-based persistence.

Technologies Used

S/W:

- Visual Studio Code
- HTML5
- CSS3
- Vanilla JavaScript
- Node.js built-in HTTP server
- JSON file storage
- Git and GitHub
- Google Chrome / Microsoft Edge
- Vercel-compatible API structure

H/W:

- Laptop or desktop system
- Intel/AMD processor
- 4 GB RAM or above
- Minimum 200 MB free storage

- Internet connection for repository and hosting access

Introduction of Topic/Project

StockPilot is a modern inventory management web application designed for small business use. The system maintains stock items, records stock-in and stock-out activity, shows low-stock alerts, presents category-wise chart analysis, and supports shared workspace access for multiple roles. Its scope includes inventory visibility, operational analytics, responsive design, smooth user experience, and simple collaboration. The project is important because it addresses practical stock-management needs through a clear, minimal, and easy-to-use digital system.

Project Architecture

The project follows a lightweight client-server architecture. The frontend layer renders the interface and interacts with backend APIs. The backend validates requests, updates inventory records, manages demo seed data, and serves the application files. The application also supports a Vercel-compatible API entry for cloud deployment.

- Presentation Layer: index.html and style.css build the inventory dashboard and responsive UI.
- Client Logic Layer: script.js handles rendering, analytics calculations, chart drawing, theme switching, and API communication.
- Server Layer: server.js serves local API routes and static assets.
- Deployment API Layer: api/index.js provides a serverless-compatible backend entry for Vercel.
- Data Layer: lib/store.js and data/store.json maintain inventory items, stock movements, and shared-access data.
- Version Control Layer: the project is stored in the GitHub repository https://github.com/vedant27-lab/stockpilot.git

Project design/functionalities/features

Module 1: Inventory Item Management

- Add new inventory items with name, SKU, category, supplier, price, opening quantity, and reorder level
- Display all items in a searchable inventory register
- Delete inventory items when required
- Show low-stock status using clear visual indicators

Module 2: Stock Movement Management

- Record stock-in and stock-out operations
- Automatically update current quantity after each movement
- Store note and time for each movement event
- Maintain recent inventory activity timeline

Module 3: Analytics and Graph Analysis

- Show summary cards for total items, total units, inventory value, and reorder count
- Visualize category-wise stock distribution using an SVG bar chart
- Show aggregate movement insights such as incoming and outgoing units

- Use richer demo data to create meaningful graphs and screenshots

Module 4: Access Sharing Module

- Create access invites for teammates
- Assign Viewer or Editor role
- Show invite token and status in the shared-access panel
- Revoke access records when needed

Module 5: UI and Accessibility Module

- Smooth and responsive layout for desktop and mobile devices
- Light and dark mode toggle at the corner
- Accessible labels, focus states, skip link, and semantic sections
- Mobile-friendly table-to-card transformation on smaller screens

Working/flowchart/algorithms

Overall Working:

1. User opens the StockPilot inventory workspace.
2. Frontend requests dashboard data from the backend API.
3. Backend loads products, stock movements, and shared-access records from the store.
4. Frontend renders summary cards, graphs, inventory table, alerts, timeline, and share list.
5. When a new inventory item is added, the frontend sends the form data to the products API.
6. When stock movement is recorded, the backend validates the quantity and updates stock accordingly.
7. When a share invite is created, the backend stores the access record with role and token.
8. Frontend reloads dashboard data and updates charts and tables immediately.

Textual Flowchart:

- Start
- Open inventory workspace
- Fetch dashboard data
- Render items, analytics, and access records
- User selects action: add item / record movement / create invite / search / delete / reset / theme switch
- Validate request
- Update store
- Refresh dashboard
- Stop

Algorithm for Stock Movement:

9. Accept selected item, movement type, quantity, and note from the user.
10. Check whether the selected item exists.
11. Validate that quantity is greater than zero.
12. If movement type is stock-out, verify stock availability.
13. Update quantity by adding or subtracting the movement amount.
14. Create a movement record with note, time, and amount value.
15. Save updated data and refresh the dashboard.

Other information

- The project includes a large 70-item demo inventory dataset for realistic testing and screenshots.
- The chart is generated without external chart libraries, using SVG and JavaScript.
- The app supports both local Node.js execution and a Vercel-compatible API route structure.
- The GitHub repository used for the project is https://github.com/vedant27-lab/stockpilot.git

Conclusion

StockPilot Inventory Management System successfully demonstrates the implementation of a dynamic web application for business use. The application combines inventory control, stock movement tracking, graph-based analytics, responsive UI, and access-sharing features in one practical system. It clearly reflects the learning and application of core web technologies in a real-world mini-project format.

Code snippets explaining features and architecture

The following code snippets are taken from the current project and show how the major features are implemented.

Snippet 1: Theme toggle handling from script.js

```
function resolvePreferredTheme() {
  const savedTheme = localStorage.getItem(THEME_KEY);
  if (savedTheme === "light" || savedTheme === "dark") {
    return savedTheme;
  }

  return window.matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light";
}

function applyTheme(theme) {
  document.documentElement.dataset.theme = theme;
  localStorage.setItem(THEME_KEY, theme);
  const darkMode = theme === "dark";
  els.themeToggle.textContent = darkMode ? "Light Mode" : "Dark Mode";
  els.themeToggle.setAttribute("aria-pressed", String(darkMode));
}

function toggleTheme() {
  applyTheme(document.documentElement.dataset.theme === "dark" ? "light" : "dark");
}
```

Explanation: This code stores and restores the selected theme, applies the theme to the document root, and keeps the floating corner toggle synchronized with the current mode.

Snippet 2: Inventory analytics calculation from script.js

```
function getAnalytics() {
  const totalUnits = state.products.reduce((sum, product) => sum + product.quantity, 0);
  const inventoryValue = state.products.reduce((sum, product) => sum + product.quantity
* product.price, 0);
  const lowStock = state.products.filter((product) => product.quantity <=
product.threshold);
  const incomingUnits = state.movements
    .filter((entry) => entry.type === "in")
    .reduce((sum, entry) => sum + entry.quantity, 0);
  const outgoingUnits = state.movements
    .filter((entry) => entry.type === "out")
    .reduce((sum, entry) => sum + entry.quantity, 0);

  const categoryMap = new Map();
  state.products.forEach((product) => {
    categoryMap.set(product.category, (categoryMap.get(product.category) || 0) +
product.quantity);
  });

  return {
    totalUnits,
    inventoryValue,
    lowStock,
    incomingUnits,
    outgoingUnits,
    categoryBreakdown: [...categoryMap.entries()],
  };
}
```

Explanation: This snippet calculates total units, inventory value, reorder alerts, incoming stock, outgoing stock, and category-wise data required for graph analysis.

Snippet 3: Category chart rendering from script.js

```
function renderCategoryChart() {
  const entries = getAnalytics().categoryBreakdown;
  els.categoryChart.innerHTML = "";

  if (!entries.length) {
    els.categoryChart.innerHTML = `
      <text x="50%" y="50%" text-anchor="middle" fill="currentColor" opacity="0.55">
        No category data available
      </text>
    `;
    return;
  }

  const maxValue = Math.max(...entries.map(([, value]) => value), 1);
  const colors = ["#d66a3c", "#3e8e7e", "#4c6edb", "#d9a441", "#9b5de5", "#2f9e44"];
  const width = 600;
  const chartHeight = 170;
  const baseY = 210;
  const barWidth = Math.max(52, Math.floor(420 / entries.length));
  const gap = 24;
  const startX = 56;

  const axis = `
    <line x1="40" y1="${baseY}" x2="${width - 20}" y2="${baseY}" stroke="currentColor"
opacity="0.2" />
    <line x1="40" y1="24" x2="40" y2="${baseY}" stroke="currentColor" opacity="0.2" />
  `;

  const bars = entries
    .map(([label, value], index) => {
      const x = startX + index * (barWidth + gap);
      const height = Math.max(18, Math.round((value / maxValue) * chartHeight));
      const y = baseY - height;
      const color = colors[index % colors.length];
      return `
        <rect x="${x}" y="${y}" width="${barWidth}" height="${height}" rx="18"
fill="${color}" opacity="0.88"></rect>
        <text x="${x + barWidth / 2}" y="${y - 8}" text-anchor="middle" font-size="14"
fill="currentColor">${value}</text>
        <text x="${x + barWidth / 2}" y="${baseY + 18}" text-anchor="middle" font-
size="13" fill="currentColor">${label}</text>
      `;
    })
    .join("");

  els.categoryChart.innerHTML = axis + bars;
}
```

Explanation: This feature generates the category stock bar chart using SVG elements, without any complex framework or chart library.

Snippet 4: Inventory movement API handling from server.js

```
  if (req.method === "POST" && url.pathname === "/api/movements") {
    const payload = await readJsonBody(req);
    const error = validateMovement(payload);
    if (error) {
```

```
        sendJson(res, 400, { message: error });
        return;
      }

      const store = await readStore();
      const product = store.products.find((item) => item.id === payload.productId);

      if (!product) {
        sendJson(res, 404, { message: "Selected inventory item was not found." });
        return;
      }

      const quantity = Number(payload.quantity);

      if (payload.type === "out" && quantity > product.quantity) {
        sendJson(res, 400, { message: "Outgoing quantity exceeds available stock." });
        return;
      }

      product.quantity += payload.type === "in" ? quantity : -quantity;
      product.updatedAt = new Date().toISOString();

      store.movements.unshift({
        id: randomUUID(),
        type: payload.type,
        productId: product.id,
        productName: product.name,
        quantity,
        note: payload.note.trim(),
        amount: product.price * quantity,
        createdAt: new Date().toISOString(),
      });

      await writeStore(store);
      sendJson(res, 201, { message: "Inventory movement recorded." });
      return;
    }
```

Explanation: This backend route validates stock movement requests, ensures stock cannot go negative, updates the item quantity, and stores a movement record for the timeline and analytics.

Snippet 5: Seed catalog generation from lib/store.js

```
function createSeedState() {
  const now = Date.now();
  const baseProducts = [
    ["Premium Notebook", "STK-101", "Stationery", "Campus Paper Co.", 299, 34, 10],
    ["Wireless Mouse", "ELE-220", "Electronics", "Pixel Devices", 799, 7, 8],
    ["Ceramic Coffee Mug", "HOM-045", "Home Goods", "Daily Living", 249, 18, 6],
    ["Desk Study Lamp", "LGT-310", "Lighting", "BrightNest", 1299, 12, 4],
    ["Steel Water Bottle", "LIF-125", "Lifestyle", "Urban Carry", 499, 29, 9],
    ["Portable Bluetooth Speaker", "ELE-415", "Electronics", "SoundMint", 1899, 5, 6],
    ["Document File Folder", "STK-145", "Stationery", "Campus Paper Co.", 149, 42, 12],
    ["Mini Desk Planter", "DEC-212", "Decor", "LeafLine Studio", 349, 9, 5],
    ["Wireless Keyboard", "ELE-305", "Electronics", "Pixel Devices", 1499, 14, 5],
    ["Whiteboard Marker Pack", "STK-188", "Stationery", "WriteRight Supply", 219, 48,
15],
  ];

  const catalogTemplates = [
    ["A4 Spiral Notebook", "Stationery", "Campus Paper Co.", 179, 52, 14],
    ["Gel Pen Set", "Stationery", "WriteRight Supply", 129, 63, 18],
```

```
    ["Sticky Notes Pad", "Stationery", "WriteRight Supply", 89, 57, 16],
    ["Sketch Book", "Stationery", "Campus Paper Co.", 199, 28, 8],
    ["Ring Binder", "Stationery", "Campus Paper Co.", 169, 31, 10],
    ["USB Flash Drive 64GB", "Electronics", "ByteKart", 649, 19, 6],
    ["Wireless Earbuds", "Electronics", "SoundMint", 2199, 11, 5],
    ["Laptop Stand", "Electronics", "Pixel Devices", 999, 16, 6],
    ["HD Webcam", "Electronics", "VisionCore", 1599, 8, 4],
    ["Phone Charging Cable", "Electronics", "ByteKart", 299, 37, 12],
    ["Ceramic Vase", "Home Goods", "Daily Living", 599, 17, 5],
    ["Cotton Cushion Cover", "Home Goods", "Daily Living", 349, 23, 7],
    ["Storage Basket", "Home Goods", "Daily Living", 449, 14, 5],
    ["Wall Clock", "Home Goods", "Daily Living", 899, 10, 4],
    ["Cutlery Organizer", "Home Goods", "Daily Living", 399, 21, 7],
    ["LED Night Lamp", "Lighting", "BrightNest", 699, 20, 6],
    ["Table Lantern", "Lighting", "BrightNest", 1099, 9, 4],
    ["Decor String Lights", "Lighting", "BrightNest", 549, 25, 8],
    ["Desk Accent Light", "Lighting", "BrightNest", 899, 13, 4],
    ["Rechargeable Torch", "Lighting", "BrightNest", 749, 18, 6],
    ["Yoga Bottle", "Lifestyle", "Urban Carry", 429, 24, 8],
    ["Canvas Tote Bag", "Lifestyle", "Urban Carry", 599, 15, 5],
    ["Travel Pouch", "Lifestyle", "Urban Carry", 379, 26, 8],
    ["Desk Calendar", "Lifestyle", "Urban Carry", 219, 32, 11],
    ["Minimal Wrist Journal", "Lifestyle", "Urban Carry", 329, 20, 7],
    ["Photo Frame", "Decor", "LeafLine Studio", 459, 18, 6],
    ["Scented Candle", "Decor", "LeafLine Studio", 349, 27, 9],
    ["Wall Shelf Accent", "Decor", "LeafLine Studio", 799, 7, 4],
    ["Marble Coaster Set", "Decor", "LeafLine Studio", 529, 12, 4],
    ["Mini Art Print", "Decor", "LeafLine Studio", 299, 30, 10],
  ];

  const generatedProducts = catalogTemplates.flatMap((template, templateIndex) => {
    return [1, 2].map((variant) => {
      const [name, category, supplier, price, quantity, threshold] = template;
      const id = randomUUID();
      const skuPrefix = category.slice(0, 3).toUpperCase();
      return {
        id,
        name: `${name} ${variant === 1 ? "Standard" : "Plus"}`,
        sku: `${skuPrefix}-${templateIndex + 1}${variant}`,
        category,
        supplier,
        price: price + (variant - 1) * 80,
        quantity: Math.max(4, quantity - variant * 2 + (templateIndex % 5)),
        threshold: Math.max(4, threshold),
        updatedAt: new Date(now - (templateIndex + variant) * 3600000).toISOString(),
      };
    });
  });

  const products = [...baseProducts.map([name, sku, category, supplier, price,
quantity, threshold]) => ({
    id: randomUUID(),
    name,
    sku,
    category,
    supplier,
    price,
    quantity,
    threshold,
    updatedAt: new Date().toISOString(),
  })), ...generatedProducts];
```

```
  const productByName = new Map(products.map((product) => [product.name, product]));
  const movementSeed = [
    ["Premium Notebook", "out", 2, "Front counter order for college students", 0.5],
    ["Ceramic Coffee Mug", "in", 5, "Supplier restock received in the morning", 4],
    ["Portable Bluetooth Speaker", "out", 3, "Weekend promotion bundle dispatch", 8],
    ["Whiteboard Marker Pack", "in", 18, "Bulk stationery refill for workspace demand",
26],
    ["Wireless Mouse", "out", 2, "Office setup order for startup client", 32],
    ["Steel Water Bottle", "in", 12, "New shipment added to warehouse", 40],
    ["Document File Folder", "out", 10, "Institutional stationery order packed", 52],
    ["Mini Desk Planter", "out", 4, "Lifestyle shelf sale", 68],
    ["Desk Study Lamp", "in", 6, "Premium lighting stock intake", 76],
    ["A4 Spiral Notebook Standard", "out", 11, "Semester opening stationery rush", 82],
    ["Gel Pen Set Plus", "out", 7, "Back-to-office desk refill purchase", 90],
    ["USB Flash Drive 64GB Standard", "in", 9, "Electronics restock batch arrival", 98],
    ["Wireless Earbuds Plus", "out", 4, "Gift bundle checkout", 106],
    ["Photo Frame Standard", "out", 6, "Home styling display sale", 118],
    ["LED Night Lamp Plus", "in", 8, "Lighting collection refresh", 126],
    ["Canvas Tote Bag Standard", "out", 5, "Lifestyle weekend campaign order", 134],
    ["Storage Basket Plus", "in", 7, "Warehouse replenishment cycle", 144],
    ["Scented Candle Standard", "out", 9, "Decor aisle festive sale", 156],
  ];

  const movements = movementSeed
    .map(([name, type, quantity, note, hoursAgo]) => {
      const product = productByName.get(name);
      if (!product) {
        return null;
      }

      return {
        id: randomUUID(),
        type,
        productId: product.id,
        productName: product.name,
        quantity,
        note,
        amount: product.price * quantity,
        createdAt: new Date(now - hoursAgo * 60 * 60 * 1000).toISOString(),
      };
    })
    .filter(Boolean);

  return {
    products,
    movements,
    shares: [
      {
        id: randomUUID(),
        name: "Store Manager",
        email: "manager@stockpilot.demo",
        role: "Editor",
        token: "INV-" + randomUUID().slice(0, 8).toUpperCase(),
        status: "Active",
        createdAt: new Date().toISOString(),
      },
      {
        id: randomUUID(),
        name: "Warehouse Lead",
        email: "warehouse@stockpilot.demo",
```

```
          role: "Editor",
          token: "INV-" + randomUUID().slice(0, 8).toUpperCase(),
          status: "Active",
          createdAt: new Date(now - 18 * 60 * 60 * 1000).toISOString(),
        },
        {
          id: randomUUID(),
          name: "Accounts Reviewer",
          email: "accounts@stockpilot.demo",
          role: "Viewer",
          token: "INV-" + randomUUID().slice(0, 8).toUpperCase(),
          status: "Active",
          createdAt: new Date(now - 42 * 60 * 60 * 1000).toISOString(),
        },
      ],
    };
  }
```

Explanation: This section creates the rich demo dataset used to populate the application with many inventory items, movement events, and share records for testing and screenshots.

Snippet 6: Main interface structure from index.html

```
<body>
  <a class="skip-link" href="#main-content">Skip to main content</a>

  <button
    class="theme-fab"
    id="theme-toggle"
    type="button"
    aria-pressed="false"
    aria-label="Switch between light mode and dark mode"
  >
    Dark Mode
  </button>

  <div class="page-shell">
    <header class="hero" aria-label="Inventory management overview">
      <div class="hero__content">
        <p class="eyebrow">WTL Mini Project</p>
        <h1>Inventory management with analytics and shared access.</h1>
        <p class="hero__text">
          Track stock, record incoming and outgoing movements, review category trends,
and
          share controlled workspace access with teammates from one polished dashboard.
        </p>
        <div class="hero__actions">
          <a class="button button--primary" href="#inventory-section">Manage
Inventory</a>
          <a class="button button--ghost" href="#analytics-section">View Analytics</a>
          <button class="button button--ghost" id="reset-demo" type="button">Reset Demo
Data</button>
        </div>
      </div>

      <aside class="hero__panel" aria-label="Workspace highlights">
        <div class="hero-card">
          <span>Workspace Type</span>
          <strong>Normal inventory management app</strong>
        </div>
        <div class="hero-card">
          <span>Insights</span>
```

```
            <strong>Live stock graphs and category analysis</strong>
          </div>
          <div class="hero-card">
            <span>Collaboration</span>
            <strong>Viewer and editor access sharing</strong>
          </div>
        </aside>
      </header>

      <main id="main-content">
        <section class="status-banner" aria-live="polite" id="status-banner">
          Loading workspace data...
        </section>

        <section class="stats-grid" aria-label="Inventory summary">
          <article class="stat-card">
            <span>Active Items</span>
            <strong id="total-products">0</strong>
            <small>Tracked inventory records</small>
          </article>
          <article class="stat-card">
            <span>Total Units</span>
            <strong id="total-units">0</strong>
            <small>Stock currently available</small>
          </article>
          <article class="stat-card">
            <span>Inventory Value</span>
            <strong id="inventory-value">Rs 0</strong>
            <small>Estimated current stock value</small>
          </article>
          <article class="stat-card stat-card--warning">
            <span>Reorder Needed</span>
            <strong id="low-stock-count">0</strong>
            <small>Items below the threshold</small>
          </article>
        </section>

        <section class="analytics-grid" id="analytics-section" aria-label="Analytics">
          <article class="panel panel--wide">
            <div class="panel__header panel__header--split">
              <div>
                <p class="panel__label">Analytics</p>
                <h2>Category Stock Analysis</h2>
              </div>
              <p class="panel__caption">Current units by category</p>
            </div>
            <div class="chart-surface">
              <svg id="category-chart" class="chart" viewBox="0 0 600 260" role="img"
aria-label="Bar chart showing stock by category"></svg>
            </div>
          </article>

          <article class="panel">
            <div class="panel__header">
              <div>
                <p class="panel__label">Movement Trends</p>
                <h2>Stock Flow</h2>
              </div>
            </div>
            <div id="movement-summary" class="metric-stack"></div>
          </article>
```

```
      </section>

      <section class="grid-layout" id="inventory-section">
        <article class="panel" aria-labelledby="add-product-title">
          <div class="panel__header">
            <div>
              <p class="panel__label">Inventory</p>
              <h2 id="add-product-title">Add Inventory Item</h2>
            </div>
          </div>
          <form id="product-form" class="form-grid">
            <label for="product-name">Item Name</label>
            <input type="text" id="product-name" placeholder="Premium Notebook"
required>

            <label for="product-sku">SKU</label>
            <input type="text" id="product-sku" placeholder="STK-101" required>

            <label for="product-category">Category</label>
            <input type="text" id="product-category" placeholder="Stationery" required>

            <label for="product-supplier">Supplier</label>
            <input type="text" id="product-supplier" placeholder="Campus Paper Co."
required>

            <label for="product-price">Unit Price (Rs)</label>
            <input type="number" id="product-price" min="1" step="1" placeholder="299"
required>

            <label for="product-quantity">Opening Quantity</label>
            <input type="number" id="product-quantity" min="0" step="1" placeholder="40"
required>

            <label class="form-grid__full" for="product-threshold">Reorder Level</label>
            <input class="form-grid__full" type="number" id="product-threshold" min="1"
step="1" placeholder="8" required>

            <div class="form-actions form-grid__full">
              <button class="button button--primary" type="submit">Save Inventory
Item</button>
            </div>
          </form>
        </article>

        <article class="panel" aria-labelledby="movement-title">
          <div class="panel__header">
            <div>
              <p class="panel__label">Stock Flow</p>
              <h2 id="movement-title">Record Inventory Movement</h2>
            </div>
          </div>
          <form id="movement-form" class="form-grid">
            <label class="form-grid__full" for="movement-product">Select Item</label>
            <select id="movement-product" class="form-grid__full" required></select>

            <label for="movement-type">Movement Type</label>
            <select id="movement-type" required>
              <option value="in">Stock In</option>
              <option value="out">Stock Out</option>
            </select>
```

```
            <label for="movement-quantity">Quantity</label>
            <input type="number" id="movement-quantity" min="1" step="1" placeholder="5"
required>

            <label class="form-grid__full" for="movement-note">Note</label>
            <input class="form-grid__full" type="text" id="movement-note"
placeholder="Supplier delivery or counter issue" required>

            <div class="form-actions form-grid__full">
              <button class="button button--primary" type="submit">Save
Movement</button>
            </div>
          </form>
        </article>
      </section>

      <section class="workspace-grid">
        <article class="panel panel--wide">
          <div class="panel__header panel__header--split">
            <div>
              <p class="panel__label">Inventory Register</p>
              <h2>Items in Stock</h2>
            </div>
            <div class="search-box">
              <label for="search-input">Search inventory</label>
              <input type="search" id="search-input" placeholder="Search name, SKU,
category, supplier">
            </div>
          </div>
          <div class="table-wrap">
            <table>
              <caption class="sr-only">Current inventory list</caption>
              <thead>
                <tr>
                  <th scope="col">Item</th>
                  <th scope="col">SKU</th>
                  <th scope="col">Supplier</th>
                  <th scope="col">Units</th>
                  <th scope="col">Value</th>
                  <th scope="col">Status</th>
                  <th scope="col">Action</th>
                </tr>
              </thead>
              <tbody id="inventory-table">></tbody>
            </table>
          </div>
        </article>

        <article class="panel">
          <div class="panel__header">
            <div>
              <p class="panel__label">Access Share</p>
              <h2>Invite Teammates</h2>
            </div>
          </div>
          <form id="share-form" class="form-grid">
            <label class="form-grid__full" for="share-name">Teammate Name</label>
            <input class="form-grid__full" type="text" id="share-name"
placeholder="Aditi Shah" required>

            <label class="form-grid__full" for="share-email">Email</label>
```

```
            <input class="form-grid__full" type="email" id="share-email"
placeholder="aditi@example.com" required>

            <label class="form-grid__full" for="share-role">Access Role</label>
            <select class="form-grid__full" id="share-role" required>
              <option value="Viewer">Viewer</option>
              <option value="Editor">Editor</option>
            </select>

            <div class="form-actions form-grid__full">
              <button class="button button--primary" type="submit">Create Share
Access</button>
            </div>
          </form>
          <div id="share-list" class="stack-list"></div>
        </article>
      </section>

      <section class="workspace-grid">
        <article class="panel panel--history" aria-labelledby="movement-history-title">
          <div class="panel__header">
            <div>
              <p class="panel__label">Timeline</p>
              <h2 id="movement-history-title">Recent Inventory Activity</h2>
            </div>
          </div>
          <div id="movement-history" class="sales-history"></div>
        </article>

        <article class="panel" aria-labelledby="restock-title">
          <div class="panel__header">
            <div>
              <p class="panel__label">Restock Queue</p>
              <h2 id="restock-title">Priority Alerts</h2>
            </div>
          </div>
          <div id="alert-list" class="stack-list"></div>
        </article>
      </section>
    </main>
  </div>

  <template id="empty-state-template">
    <div class="empty-state">
      <strong>No data available</strong>
      <p>Add an inventory item to populate the workspace.</p>
    </div>
  </template>
```

Explanation: This markup defines the major UI architecture including the hero section, summary cards, analytics panel, inventory forms, register table, movement timeline, and access-sharing panel.

Code

All code files with their names and proper extensions are listed below.

File Name	Extension	Purpose
index.html	.html	Main inventory workspace layout

style.css	.css	Smooth responsive styling, theme system, chart and card styles
script.js	.js	Frontend logic for inventory, analytics, charts, and access sharing
server.js	.js	Local Node.js backend server and API routing
api/index.js	.js	Vercel-compatible serverless API entry
lib/store.js	.js	Seed data generation and shared data store utilities
data/store.json	.json	Persistent demo dataset for inventory, movements, and shares
package.json	.json	Project metadata and start script
vercel.json	.json	Vercel deployment rewrite configuration
README.md	.md	Project setup and deployment notes

Output/screenshots

The screenshots below are taken from the current inventory-management version of the website.

Screenshot: image.png

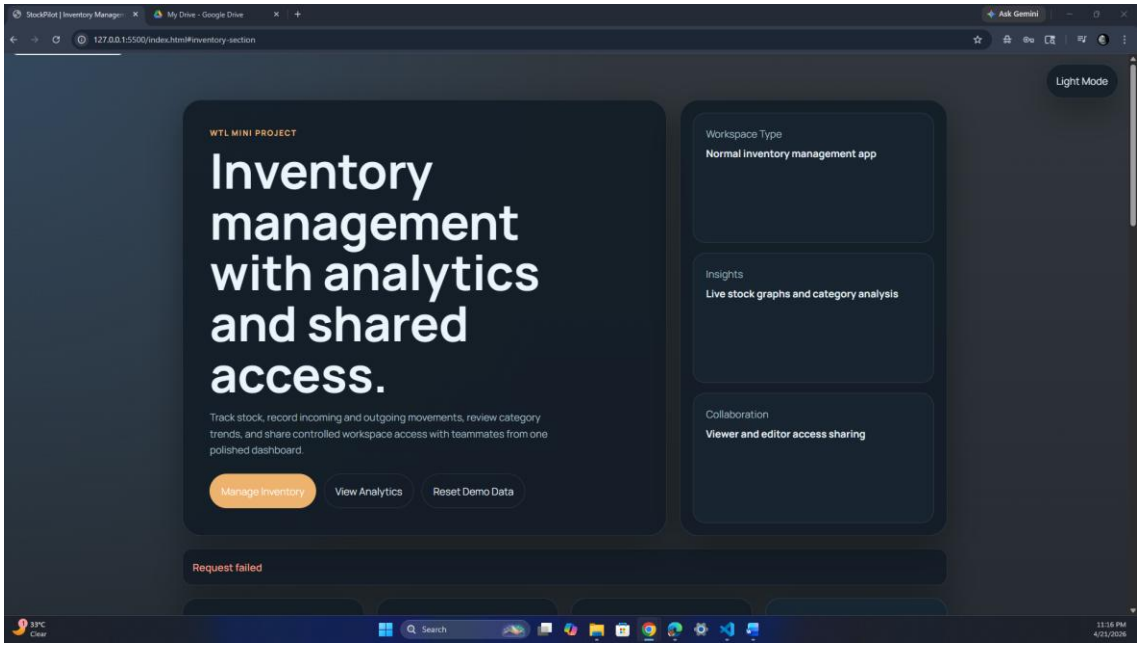

Figure 1: Website screenshot

Screenshot: image copy.png

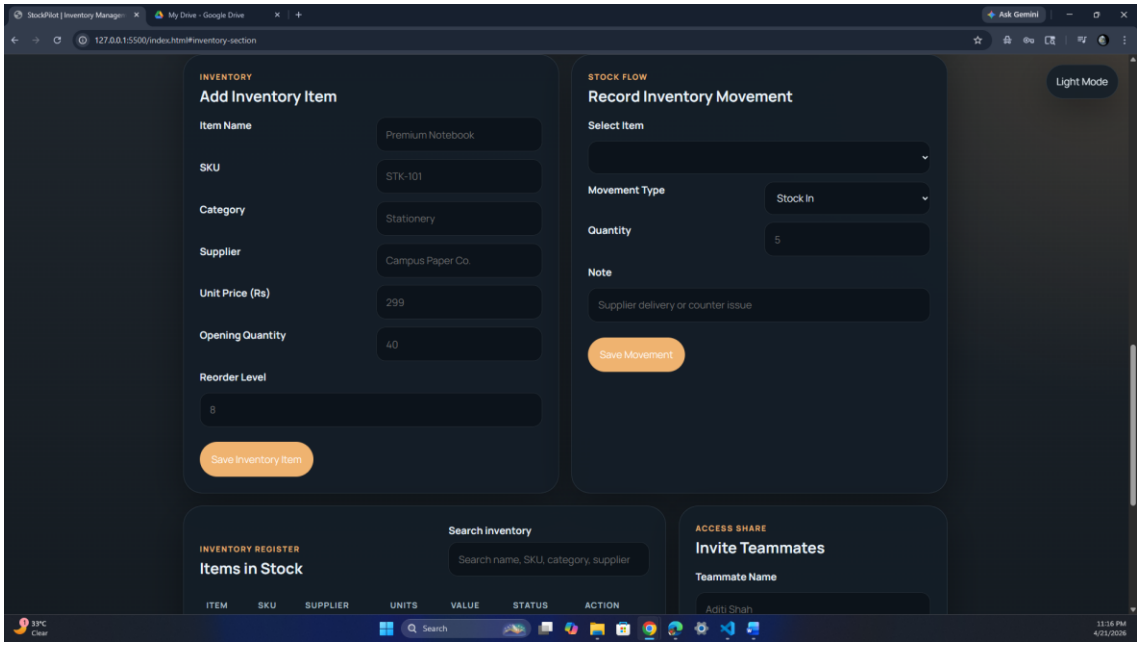

Figure 2: Website screenshot

Screenshot: image copy 2.png

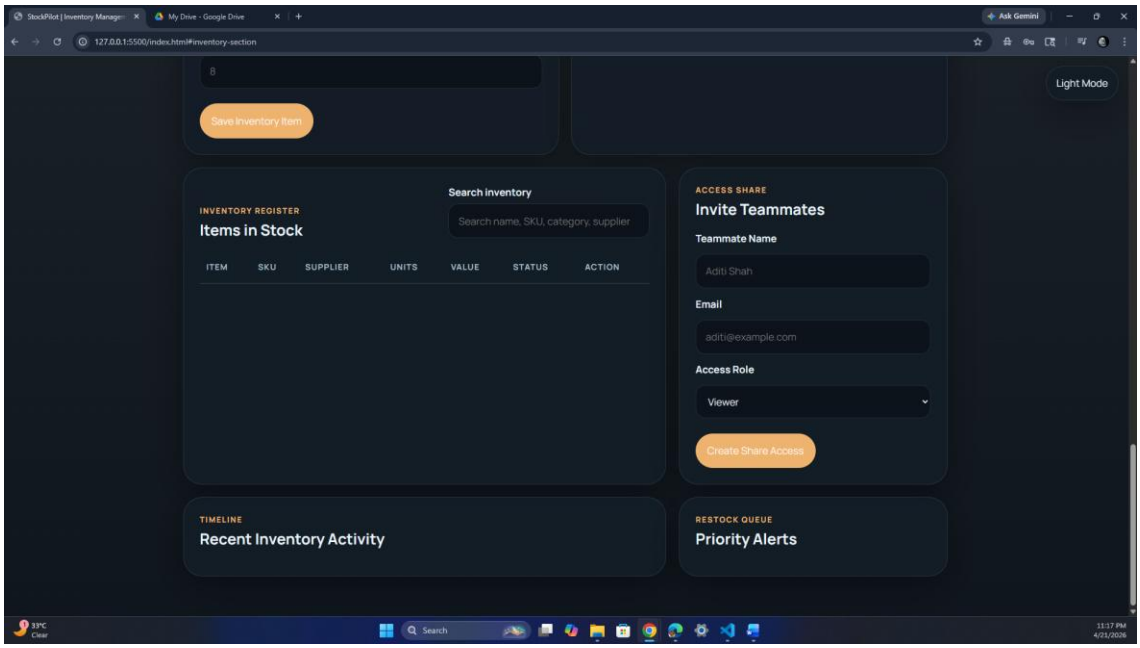

Figure 3: Website screenshot

Suggested screenshot coverage:

16. Dashboard with summary cards and chart analysis
17. Inventory add form and stock movement form
18. Inventory register with many items
19. Reorder alert section

Repository Reference

GitHub Repository: https://github.com/vedant27-lab/stockpilot.git

Local Project Folder: WTL Mini-project